

Advanced Data Structure and Algorithms (CSL7560)

- Vimal Raj Sharma

Lecture 1

Course Overview, Administrative Details, Introduction

Prerequisites

Prerequisites

This course assumes the knowledge of:

Prerequisites

This course assumes the knowledge of:

- Order notations: O , Θ , Ω .

Prerequisites

This course assumes the knowledge of:

- Order notations: O , Θ , Ω .
- Data structures such as Stack, Queue, Linked list, etc.

Prerequisites

This course assumes the knowledge of:

- Order notations: O , Θ , Ω .
- Data structures such as Stack, Queue, Linked list, etc.
- Sorting & searching algorithm such as Insertion sort, Merge sort, Quick sort, Binary search, etc.

Prerequisites

This course assumes the knowledge of:

- Order notations: O , Θ , Ω .
- Data structures such as Stack, Queue, Linked list, etc.
- Sorting & searching algorithm such as Insertion sort, Merge sort, Quick sort, Binary search, etc.
- Basics of Graphs, BFS, DFS, etc.

Prerequisites

This course assumes the knowledge of:

- Order notations: O , Θ , Ω .
- Data structures such as Stack, Queue, Linked list, etc.
- Sorting & searching algorithm such as Insertion sort, Merge sort, Quick sort, Binary search, etc.
- Basics of Graphs, BFS, DFS, etc.

A wonderful course through which you can revise the above topics: [Naveen Garg's course](#).

Tentative Course Plan

Tentative Course Plan

- **Data Structure:** BST, Red-Black Trees, Augmenting Trees, Interval Trees, Disjoint-Set Union.

Tentative Course Plan

- **Data Structure:** BST, Red-Black Trees, Augmenting Trees, Interval Trees, Disjoint-Set Union.
- **Graphs:** Introduction, Dijkstra, Network Flow, Ford-Fulkerson Method, Max-flow Min-cut Theorem, Edmond-Karp Algorithm, Applications to Bipartite Matching and Disjoint Path Problems.

Tentative Course Plan

- **Data Structure:** BST, Red-Black Trees, Augmenting Trees, Interval Trees, Disjoint-Set Union.
- **Graphs:** Introduction, Dijkstra, Network Flow, Ford-Fulkerson Method, Max-flow Min-cut Theorem, Edmond-Karp Algorithm, Applications to Bipartite Matching and Disjoint Path Problems.
- **Divide and Conquer:** Closest Pair, Fast Fourier Transform.

Tentative Course Plan

- **Data Structure:** BST, Red-Black Trees, Augmenting Trees, Interval Trees, Disjoint-Set Union.
- **Graphs:** Introduction, Dijkstra, Network Flow, Ford-Fulkerson Method, Max-flow Min-cut Theorem, Edmond-Karp Algorithm, Applications to Bipartite Matching and Disjoint Path Problems.
- **Divide and Conquer:** Closest Pair, Fast Fourier Transform.
- **Dynamic Programming:** Rod-Cutting, LCS, Maximum Product Subarray, Bellman-Ford.

Tentative Course Plan

- **Data Structure:** BST, Red-Black Trees, Augmenting Trees, Interval Trees, Disjoint-Set Union.
- **Graphs:** Introduction, Dijkstra, Network Flow, Ford-Fulkerson Method, Max-flow Min-cut Theorem, Edmond-Karp Algorithm, Applications to Bipartite Matching and Disjoint Path Problems.
- **Divide and Conquer:** Closest Pair, Fast Fourier Transform.
- **Dynamic Programming:** Rod-Cutting, LCS, Maximum Product Subarray, Bellman-Ford.
- **Greedy:** Interview Scheduling, Minimum Spanning Trees (with Proofs).

Tentative Course Plan

- **Data Structure:** BST, Red-Black Trees, Augmenting Trees, Interval Trees, Disjoint-Set Union.
- **Graphs:** Introduction, Dijkstra, Network Flow, Ford-Fulkerson Method, Max-flow Min-cut Theorem, Edmond-Karp Algorithm, Applications to Bipartite Matching and Disjoint Path Problems.
- **Divide and Conquer:** Closest Pair, Fast Fourier Transform.
- **Dynamic Programming:** Rod-Cutting, LCS, Maximum Product Subarray, Bellman-Ford.
- **Greedy:** Interview Scheduling, Minimum Spanning Trees (with Proofs).
- **Randomized Algorithms:** Miller-Rabin, Karger's Algorithm.

Tentative Course Plan

- **Data Structure:** BST, Red-Black Trees, Augmenting Trees, Interval Trees, Disjoint-Set Union.
- **Graphs:** Introduction, Dijkstra, Network Flow, Ford-Fulkerson Method, Max-flow Min-cut Theorem, Edmond-Karp Algorithm, Applications to Bipartite Matching and Disjoint Path Problems.
- **Divide and Conquer:** Closest Pair, Fast Fourier Transform.
- **Dynamic Programming:** Rod-Cutting, LCS, Maximum Product Subarray, Bellman-Ford.
- **Greedy:** Interview Scheduling, Minimum Spanning Trees (with Proofs).
- **Randomized Algorithms:** Miller-Rabin, Karger's Algorithm.
- **Approximation Algorithms:** Vertex Cover Problem, TSP, etc.

Tentative Course Plan

- **Data Structure:** BST, Red-Black Trees, Augmenting Trees, Interval Trees, Disjoint-Set Union.
- **Graphs:** Introduction, Dijkstra, Network Flow, Ford-Fulkerson Method, Max-flow Min-cut Theorem, Edmond-Karp Algorithm, Applications to Bipartite Matching and Disjoint Path Problems.
- **Divide and Conquer:** Closest Pair, Fast Fourier Transform.
- **Dynamic Programming:** Rod-Cutting, LCS, Maximum Product Subarray, Bellman-Ford.
- **Greedy:** Interview Scheduling, Minimum Spanning Trees (with Proofs).
- **Randomized Algorithms:** Miller-Rabin, Karger's Algorithm.
- **Approximation Algorithms:** Vertex Cover Problem, TSP, etc.
- **NP-completeness:** P, NP, NP-completeness, Reductions.

Administrative Details

Administrative Details

Grading:

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)
- 28% - Minor

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)
- 28% - Minor
- 42% - Major

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)
- 28% - Minor
- 42% - Major
- 0% - Practice problems whose variants may come in exam

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)
- 28% - Minor
- 42% - Major
- 0% - Practice problems whose variants may come in exam
- Bonus 1% - For every first catch of an error during the lectures

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)
- 28% - Minor
- 42% - Major
- 0% - Practice problems whose variants may come in exam
- Bonus 1% - For every first catch of an error during the lectures

One can get at most 5%.



Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)
- 28% - Minor
- 42% - Major
- 0% - Practice problems whose variants may come in exam
- Bonus 1% - For every first catch of an error during the lectures.



If you catch an error during a lecture, mail me your name and error after the class.

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)
- 28% - Minor
- 42% - Major
- 0% - Practice problems whose variants may come in exam
- Bonus 1% - For every first catch of an error during the lectures.

Note: No requizzes will be conducted. So try to not miss more than 1 quiz.

Administrative Details

Grading:

- 10% - Ten programming problems on [CSES](#).
- 20% - Quizzes (MCQs, T/Fs) (best 2 out of 3 quizzes)
- 28% - Minor
- 42% - Major
- 0% - Practice problems whose variants may come in exam
- Bonus 1% - For every first catch of an error during the lectures.

Note: No requizzes will be conducted. So try to not miss more than 1 quiz.

Note: Grading will be relative. Score at least **30%** to pass the course.

Administrative Details

Administrative Details

Book: Introduction to Algorithms by CLRS

Administrative Details

Book: Introduction to Algorithms by CLRS

Office Hours: Mail me to fix an appointment.

Administrative Details

Book: Introduction to Algorithms by CLRS

Office Hours: Mail me to fix an appointment.

Course Site: <http://home.iitj.ac.in/~vimalraj/courses/csl7560.html>

Administrative Details

Book: Introduction to Algorithms by CLRS

Office Hours: Mail me to fix an appointment.

Course Site: <http://home.iitj.ac.in/~vimalraj/courses/csl7560.html>

Attendance: As per the institute policy.

Administrative Details

Book: Introduction to Algorithms by CLRS

Office Hours: Mail me to fix an appointment.

Course Site: <http://home.iitj.ac.in/~vimalraj/courses/csl7560.html>

Attendance: As per the institute policy.

Note: Major cases of unfair means will be reported to the disciplinary committee.

What's a Problem?

What's a Problem?

What is a computational problem?

What's a Problem?

What is a computational problem?

Given an **input** (aka instance)

What's a Problem?

What is a computational problem?

Given an **input** (aka **instance**), a computational problem asks to return as **output** a solution

What's a Problem?

What is a computational problem?

Given an **input** (aka **instance**), a computational problem asks to return as **output** a solution satisfying some property.

What's a Problem?

What is a computational problem?

Given an **input (aka instance)**, a computational problem asks to return as **output** a solution satisfying some property.

Examples:

What's a Problem?

What is a computational problem?

Given an **input (aka instance)**, a computational problem asks to return as **output** a solution satisfying some property.

Examples:

Prime:

What's a Problem?

What is a computational problem?

Given an **input (aka instance)**, a computational problem asks to return as **output** a solution satisfying some property.

Examples:

Prime:

Input: A positive integer *n* .

What's a Problem?

What is a computational problem?

Given an **input** (aka **instance**), a computational problem asks to return as **output** a solution satisfying some property.

Examples:

Prime:

Input: A positive integer n .

Output: Find whether n is prime.

What's a Problem?

What is a computational problem?

Given an **input (aka instance)**, a computational problem asks to return as **output** a solution satisfying some property.

Examples:

Prime:

Input: A positive integer *n*.

Output: Find whether *n* is prime.

Sorting:

What's a Problem?

What is a computational problem?

Given an **input (aka instance)**, a computational problem asks to return as **output** a solution satisfying some property.

Examples:

Prime:

Input: A positive integer n .

Output: Find whether n is prime.

Sorting:

Input: A sequence of n numbers $(a_1, a_2, \dots, a_n)$.

What's a Problem?

What is a computational problem?

Given an **input (aka instance)**, a computational problem asks to return as **output** a solution satisfying some property.

Examples:

Prime:

Input: A positive integer n .

Output: Find whether n is prime.

Sorting:

Input: A sequence of n numbers $(a_1, a_2, \dots, a_n)$.

Output: A permutation $(a'_1, a'_2, \dots, a'_n)$ of the input sequence s.t. $a'_1 \leq a'_2 \dots \leq a'_n$.

What's an Algorithm?

What's an Algorithm?

What is an Algorithm?

What's an Algorithm?

What is an Algorithm?

An algorithm is a **sequence** of **computational steps** that transforms the **input** into the **output**.

What's an Algorithm?

What is an Algorithm?

An algorithm is a **sequence** of **computational steps** that transforms the **input** into the **output**.

Algorithm for a Problem:

What's an Algorithm?

What is an Algorithm?

An algorithm is a **sequence** of **computational steps** that transforms the **input** into the **output**.

Algorithm for a Problem:

An algorithm for a (computational) problem is **correct**

What's an Algorithm?

What is an Algorithm?

An algorithm is a **sequence** of **computational steps** that transforms the **input** into the **output**.

Algorithm for a Problem:

An algorithm for a (computational) problem is **correct**, if for every problem instance it **halts** and

What's an Algorithm?

What is an Algorithm?

An algorithm is a **sequence** of **computational steps** that transforms the **input** into the **output**.

Algorithm for a Problem:

An algorithm for a (computational) problem is **correct**, if for every problem instance it **halts** and outputs the **correct solution**.

What's an Algorithm?

What's an Algorithm?

Example: Algorithm to solve *Prime*.

What's an Algorithm?

Example: Algorithm to solve *Prime*.

PRIME(n):

What's an Algorithm?

Example: Algorithm to solve *Prime*.

PRIME(n):

1. if $n == 1$

What's an Algorithm?

Example: Algorithm to solve *Prime*.

PRIME(n):

1. if $n == 1$
2. return No

What's an Algorithm?

Example: Algorithm to solve *Prime*.

PRIME(n):

1. if $n == 1$
2. return No
3. for $i = 2$ to $n - 1$

What's an Algorithm?

Example: Algorithm to solve *Prime*.

PRIME(n):

1. if $n == 1$
2. return No
3. for $i = 2$ to $n - 1$
4. if $n \% i == 0$

What's an Algorithm?

Example: Algorithm to solve *Prime*.

PRIME(n):

1. if $n == 1$
2. return No
3. for $i = 2$ to $n - 1$
4. if $n \% i == 0$
5. return No

What's an Algorithm?

Example: Algorithm to solve *Prime*.

PRIME(n):

1. if $n == 1$
2. return No
3. for $i = 2$ to $n - 1$
4. if $n \% i == 0$
5. return No
6. return Yes

What's a Data Structure?

What's a Data Structure?

What is a Data Structure?

What's a Data Structure?

What is a Data Structure?

A data structure is a way to **store** and **organise** data in order to facilitate **access** & **modifications**.

What's a Data Structure?

What is a Data Structure?

A data structure is a way to **store** and **organise** data in order to facilitate **access** & **modifications**.

Note: Using appropriate data structures are an important part of algorithm design.

Goals of this Course

Goals of this Course

1) Designing algorithms to solve computational problems.

Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

$(a, b):$

Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

$(a, b):$

Assume a, b are non-negative integers and both are not zeroes.



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

```
(a, b):  
1.  if b == 0
```

Assume a, b are non-negative integers and both are not zeroes.



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

```
(a, b):  
1.  if  $b == 0$   
2.    return  $a$ 
```

Assume a, b are non-negative integers and both are not zeroes.



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

```
(a, b):  
1.  if  $b == 0$   
2.    return  $a$   
3.  else
```

Assume a, b are non-negative integers and both are not zeroes.



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

```
(a, b):  
1.  if  $b == 0$   
2.    return  $a$   
3.  else  
4.    for  $i = \min(a, b)$  to 1
```

Assume a, b are non-negative integers and both are not zeroes.



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

```
(a, b):  
1.  if  $b == 0$   
2.    return  $a$   
3.  else  
4.    for  $i = \min(a, b)$  to 1  
5.      if  $a \% i == 0$  and  $b \% i == 0$ 
```

Assume a, b are non-negative integers and both are not zeroes.



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

```
(a, b):  
1.  if  $b == 0$   
2.      return  $a$   
3.  else  
4.      for  $i = \min(a, b)$  to 1  
5.          if  $a \% i == 0$  and  $b \% i == 0$   
6.              return  $i$ 
```

Assume a, b are non-negative integers and both are not zeroes.



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

GCD(a, b):

```
1.  if  $b == 0$ 
2.    return  $a$ 
3.  else
4.    for  $i = \min(a, b)$  to 1
5.      if  $a \% i == 0$  and  $b \% i == 0$ 
6.        return  $i$ 
```

Assume a, b are non-negative integers and both are not zeroes.



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

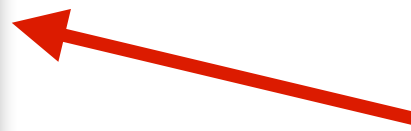
GCD(a, b):

```
1.  if  $b == 0$ 
2.    return  $a$ 
3.  else
4.    for  $i = \min(a, b)$  to 1
5.      if  $a \% i == 0$  and  $b \% i == 0$ 
6.        return  $i$ 
```

Assume a, b are non-negative integers and both are not zeroes.



Correctness is obvious



Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

GCD(a, b):

1. **if $b == 0$**
2. **return a**
3. **else**
4. **return GCD($b, a \% b$)**

Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

GCD(a, b):

```
1.  if  $b == 0$   
2.    return  $a$   
3.  else  
4.    return GCD( $b, a \% b$ )
```

← Requires proof of correctness.

Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

GCD(a, b):

```
1.  if  $b == 0$   
2.    return  $a$   
3.  else  
4.    return GCD( $b, a \% b$ )
```

← Requires proof of correctness.

- 3) Argue about the **efficiency** of the algorithms.

Goals of this Course

- 1) Designing algorithms to solve computational problems.
- 2) Prove **correctness** of algorithms.

GCD(a, b):

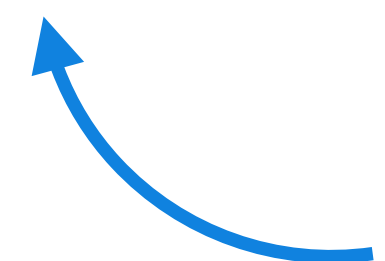
```
1.  if  $b == 0$   
2.    return  $a$   
3.  else  
4.    return GCD( $b, a \% b$ )
```

Requires proof of correctness.



- 3) Argue about the **efficiency** of the algorithms.

Will be mostly concerned with time and space.



Runtime of an Algorithm

Runtime of an Algorithm

How to measure the running time of an algorithm?

Runtime of an Algorithm

How to measure the running time of an algorithm?

- Code it and calculate its runtime on different inputs.

Runtime of an Algorithm

How to measure the running time of an algorithm?

- Code it and calculate its runtime on different inputs. 

Runtime of an Algorithm

How to measure the running time of an algorithm?

- Code it and calculate its runtime on different inputs. 
- It's the **# of basic operations** it performs in the worst case as a function of the input size.

Runtime of an Algorithm

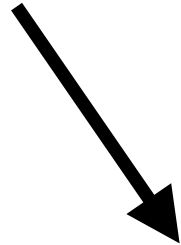
How to measure the running time of an algorithm?

- Code it and calculate its runtime on different inputs. 
- It's the # of basic operations it performs in the worst case as a function of the input size.

... such as $A[i] = 2$, $a = b + c$, $a == 0$, etc.

Runtime of an Algorithm

How to measure the running time of an algorithm?

- Code it and calculate its runtime on different inputs. 
- It's the # of basic operations it performs in the worst case as a function of the input size. 

... such as $A[i] = 2$, $a = b + c$, $a == 0$, etc.

Runtime of an Algorithm

Runtime of an Algorithm

Examples:

Runtime of an Algorithm

Examples:

- Insertion sort's runtime is $\Theta(n^2)$.

Runtime of an Algorithm

Examples:

- Insertion sort's runtime is $\Theta(n^2)$.
- Merge sort's runtime is $\Theta(n \log n)$.

Runtime of an Algorithm

Examples:

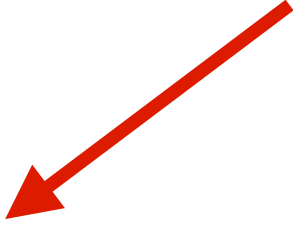
- Insertion sort's runtime is $\Theta(n^2)$.
- Merge sort's runtime is $\Theta(n \log n)$.

On large enough inputs of length n , Insertion sort takes around cn^2 basic operations.



Runtime of an Algorithm

Examples:

- Insertion sort's runtime is $\Theta(n^2)$.

On large enough inputs of length n , Insertion sort takes around cn^2 basic operations.
- Merge sort's runtime is $\Theta(n \log n)$.

On large enough inputs of length n , Merge sort takes around $c'n \log n$ basic operations.